

Tower of Hanoi

Virtual Reality and Mixed Reality

Using and Programming Virtual Reality Headsets

Michael Enriquez (20246934)
Sohaib Siddique Butt (20246031)

Supervisor: Jean-Yves Didier

Master 2 in Mechatronics, Machine Vision and Artificial Intelligence

Contents

1	Introduction	2
1.1	Tower of Hanoi Rules	2
1.2	Technical Framework	3
1.2.1	Oculus Quest Hardware Utilization	3
1.2.2	Android Mobile Device Utilization	4
2	Design of the Virtual Reality and Application	4
2.1	3D Scene Architecture Asset Structure	4
2.2	Choice of Navigation Technique	8
2.3	Choice and Design of Selection and Manipulation Techniques	8
2.4	Specification of the Virtual Scene	9
2.5	Auditory Feedback Mechanisms	10
2.6	Visual Feedback Mechanisms	12
3	Implementation of the Virtual Reality and Application	13
4	Extending to Mixed Reality	16
4.1	Roomscale Calibration	16
4.2	AR/VR Mode Selection	16
4.3	The Placement Phase	17
5	Application Walkthrough	18
5.1	The Virtual Reality (VR) Workflow	18
5.2	The Mixed Reality (MR) Workflow	23
6	Conclusion	28

1 Introduction

This project focuses on the design and implementation of an immersive *Tower of Hanoi* puzzle game. Building upon the `three.js` framework, the objective is to deploy a fully interactive virtual and mixed reality experience on the Oculus Quest headset.

The user experience architecture initializes at a central decision node designed to determine the operational modality of the session. Upon launching the application, the user is presented with a graphical user interface (GUI), illustrated in Figure 1, which functions as the primary entry point to the system.

This interface branches the simulation into two distinct interaction paradigms, requiring the user to select their preferred immersive context before the game logic is instantiated:

1. **Immersive Virtual Reality (VR).** This mode generates a fully synthetic environment, completely occluding the physical world and maximizing user immersion within the digital scene.
2. **Mixed Reality (MR) via Passthrough.** This mode leverages the headset’s camera sensors to superimpose digital assets onto the physical environment. The Tower of Hanoi puzzle is spatially anchored to real-world surfaces such as a table or floor, blending virtual and physical contexts.

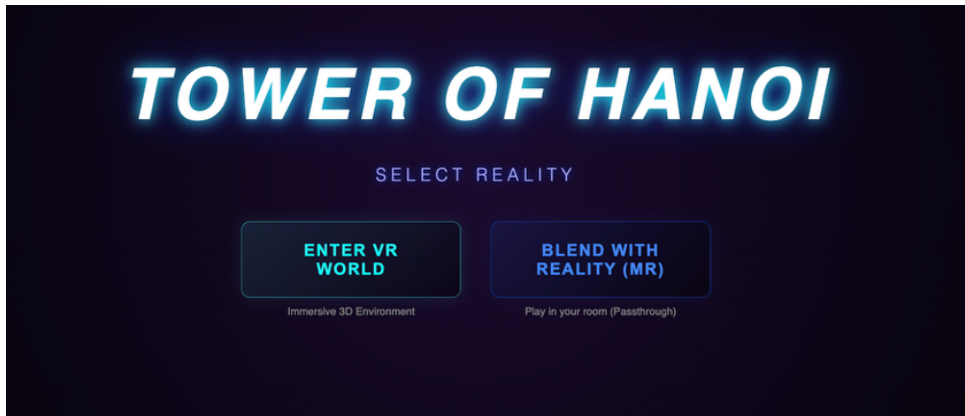


Figure 1: Graphical User Interface of the Tower of Hanoi application.

1.1 Tower of Hanoi Rules

The Tower of Hanoi is a mathematical puzzle that consists of three rods (pegs) and a number of disks of different sizes, which can slide onto any rod. The puzzle starts with the disks in a neat stack in ascending order of size on one rod, the smallest at the top, making a conical shape.

The objective of the puzzle is to move the entire stack to another rod, obeying the following simple rules:

1. Single Disk Movement. Only one disk can be moved at a time.
2. Top-Most Only. Each move consists of taking the upper disk from one of the stacks and placing it on top of another stack or on an empty rod.
3. Size Constraint. No disk may be placed on top of a smaller disk.

In this immersive adaptation, these rules are enforced programmatically. The system prevents invalid moves by automatically returning a disk to its original peg if the user attempts to place it on a smaller disk, providing immediate visual and auditory feedback (error sound and notification).

These constraints are strictly enforced by the application logic to prevent invalid game states.

1.2 Technical Framework

The application is built upon a modern web-based graphics stack designed for cross-platform compatibility and high-performance rendering in XR environments.

- Three.js. A JavaScript 3D library used to create and display animated 3D computer graphics in a web browser. It handles the scene graph, geometry creation, materials (Physical Based Rendering), and lighting.
- WebXR Device API. This API provides access to the input and output capabilities of virtual reality and augmented reality devices. It manages the stereoscopic rendering loop and tracks the position and orientation of the headset and controllers.
- WebGL. The underlying low-level API that renders interactive 2D and 3D graphics within any compatible web browser without the use of plug-ins. Three.js acts as an abstraction layer over WebGL to simplify development.

1.2.1 Oculus Quest Hardware Utilization

The project is primarily optimized for deployment on the Oculus Quest (Meta Quest) headset series. The system leverages the device’s specific hardware capabilities to drive the core user experience:

- Stereoscopic Rendering. The headset’s dual displays provide binocular vision, creating true depth perception essential for judging the spatial relationships between the disks and pegs.
- 6-DOF Tracking. The headset’s inside-out tracking system allows for 6-Degrees-of-Freedom movement. This means the user can physically walk around the virtual Tower of Hanoi (translation) and tilt or turn their head (rotation) to inspect the puzzle from any angle.

- **Passthrough Sensors (MR Mode).** In Mixed Reality mode, the application accesses the Quest's external cameras to render a real-time feed of the physical room, allowing digital assets to be overlaid securely onto the real world.
- **Touch Controllers.** The Oculus Touch controllers act as the primary input devices. The application maps the physical trigger button to the virtual "grab" action and tracks the controller's position in real-time to simulate the hand's movement within the 3D space.

1.2.2 Android Mobile Device Utilization

To ensure broader accessibility and facilitate rapid testing of the Augmented Reality (AR/MR) features, the project also supports Android mobile devices.

- **ARCore Integration.** Through the Chrome browser on Android, the WebXR API utilizes the underlying ARCore technology. This allows the device to perform environmental understanding (detecting floor planes) and motion tracking using the phone's single RGB camera and inertial sensors.
- **Monoscopic Passthrough.** Unlike the headset, the mobile device renders a monoscopic view where the 3D Tower of Hanoi is superimposed onto the live camera feed displayed on the screen.
- **Touch Interface:** The interaction model automatically adapts to the hardware. Instead of spatial controllers, the application utilizes screen-space raycasting, where a tap on the touchscreen simulates a "trigger" event, allowing the user to select and manipulate disks using standard touch gestures.

This multi-device approach demonstrates the project's adherence to Responsive XR Design principles, ensuring the experience degrades gracefully from high-end VR headsets to standard mobile smartphones.

2 Design of the Virtual Reality and Application

This section details the interaction design and technical specifications proposed for the Virtual Reality implementation of the Tower of Hanoi. The design prioritizes total immersion, isolating the user from the physical world to focus entirely on the virtual puzzle.

2.1 3D Scene Architecture Asset Structure

The virtual environment is constructed using a hierarchical scene graph where all interactive elements inherit from the `THREE.Object3D` class. The architecture is designed to

be modular, allowing the same assets to function in both Virtual Reality (VR) and Mixed Reality (MR) modes.

1. Disks. The disks are the primary interactive agents of the puzzle (shown in Figure 3). To distinguish them from the "solid" environment, they are rendered using a high-fidelity Physically Based Rendering (PBR) material that simulates the optical properties of frosted glass (codes shown in Figure 2).
 - Geometry: Each disk is generated using `THREE.CylinderGeometry`. While the height is constant (0.06 units), the radius is calculated algorithmically based on the disk's index in the stack. A high segment count (64) is used to ensure the curvature remains perfectly smooth even when viewed up close in VR.
 - Material Composition: The visual surface utilizes `THREE.MeshPhysicalMaterial`, a complex shader that simulates advanced optical properties:
 - Transmission (0.6): Allows light to pass through the object, simulating the density of frosted glass.
 - Clearcoat (1.0): Adds a second specular layer on top of the base material, mimicking a polished or varnished surface.
 - Emissive State: The material's `emissiveIntensity` is dynamic. It rests at a low value (0.4) and spikes to high intensity (2.0) when the user grabs the object, providing immediate visual feedback.

```
const diskMat = new THREE.MeshPhysicalMaterial({  
  color: col, metalness: 0.1, roughness: 0.2, transmission: 0.6,  
  thickness: 1.0, clearcoat: 1.0, emissive: col, emissiveIntensity: 0.4  
});
```

Figure 2: Disks Assets

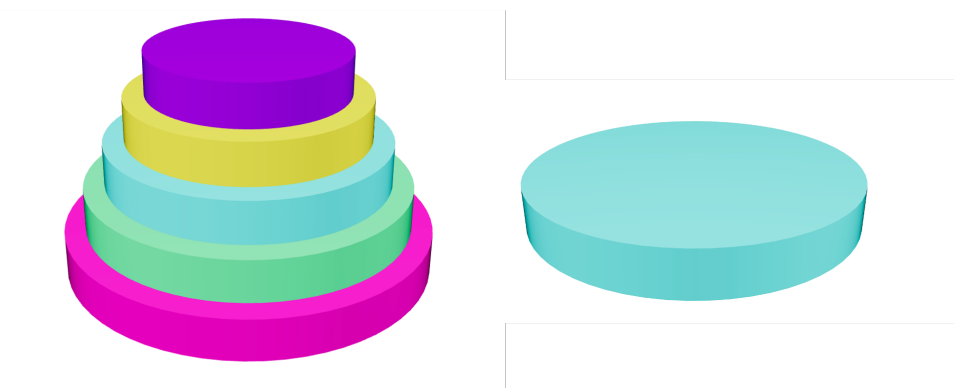


Figure 3: Tower of Hanoi Disks

2. Pegs. The pegs are designed as Composite Objects rather than single static meshes (shown in Figure 6). They utilize a Parent-Child hierarchy to separate the physical logic from the visual effects. This structure allows the visual "scanner" effect to move with the peg automatically without complex update loops.
- The Core (Physical Parent): The root of the object is a solid cylinder (0.015 radius) using a MeshStandardMaterial. This acts as the Physical Anchor. All game logic, such as distance calculations for snapping and stacking, is performed against the coordinates of this core mesh.
 - The Scanner (Visual Child): Attached as a child to the core is a slightly wider cylinder (0.02 radius).
 - Blending: It uses THREE.AdditiveBlending, which adds the color values of the texture to the background rather than obscuring it, creating a pure light/holographic effect.
 - Texture: It maps a procedural gradient texture that fades from transparent to opaque, simulating a beam of light scanning the peg.
 - The Cap Socket (Detailing): To ground the object physically, a "Socket" mesh is added at the base (where it meets the platform), and a "Cap" mesh is added to the top. These use a metallic material to contrast with the glowing scanner beam.

```
for (let i = 0; i < 3; i++) {  
  const peg = new THREE.Mesh(new THREE.CylinderGeometry(0.015, 0.015, 0.6, 32), matPegCore);  
  peg.position.set(pegPositions[i], 0.3, 0);  
  peg.castShadow = !isGhost;  
  targetGroup.add(peg);  
  
  if (!isGhost) {  
    const scanner = new THREE.Mesh(new THREE.CylinderGeometry(0.02, 0.02, 0.6, 32), matPegScan);  
    peg.add(scanner);  
    const socket = new THREE.Mesh(new THREE.CylinderGeometry(0.04, 0.05, 0.03, 32), matSocket);  
    socket.position.y = -0.28; socket.receiveShadow = true; peg.add(socket);  
    const cap = new THREE.Mesh(new THREE.CylinderGeometry(0.018, 0.018, 0.01, 32), matSocket);  
    cap.position.y = 0.3; peg.add(cap);  
    pegs.push({ mesh: peg, disks: [] });  
  }  
}
```

Figure 4: Pegs Assets

3. Platform. The game board platform is not a single monolithic mesh but a composite structure designed to separate physical grounding from visual aesthetics (shown in Figure 6). It is constructed using two distinct geometric layers that are grouped together in the scene graph.
- The Structural Base (Foundation): The core of the platform is a flattened cuboid constructed using THREE.BoxGeometry with dimensions (1.6, 0.05, 0.6).

Material. It utilizes a MeshPhysicalMaterial with high metalness (0.5) and roughness (0.4) to simulate a brushed metal surface that receives shadows from the floating disks.

Function. This mesh acts as the visual anchor, grounding the pegs in 3D space.

- The Neon Rim: Surrounding the base is a secondary, slightly larger geometry (1.65, 0.03, 0.65).

Geometry. It is also a BoxGeometry, but calculated to be exactly 0.05 units wider and deeper than the base, effectively creating a "jacket" or trim around the edges.

Material. Unlike the base, this component uses a MeshStandardMaterial with a dynamically generated Procedural Texture (Canvas Texture) applied to its emissiveMap. This allows the rim to emit light without needing an expensive post-processing glow effect.

```
const baseGeo = new THREE.BoxGeometry(1.6, 0.05, 0.6);
const base = new THREE.Mesh(baseGeo, matBase);
base.position.y = 0.025; base.receiveShadow = !isGhost;
targetGroup.add(base);

if (!isGhost) {
  const rimGeo = new THREE.BoxGeometry(1.65, 0.03, 0.65);
  const rimMat = new THREE.MeshStandardMaterial({
    color: 0x000033, emissive: 0xffffffff, emissiveMap: rimTexture, emissiveIntensity: 2.0
  });
  const rim = new THREE.Mesh(rimGeo, rimMat);
  rim.position.y = 0.01;
  targetGroup.add(rim);
}
```

Figure 5: Platform Assets

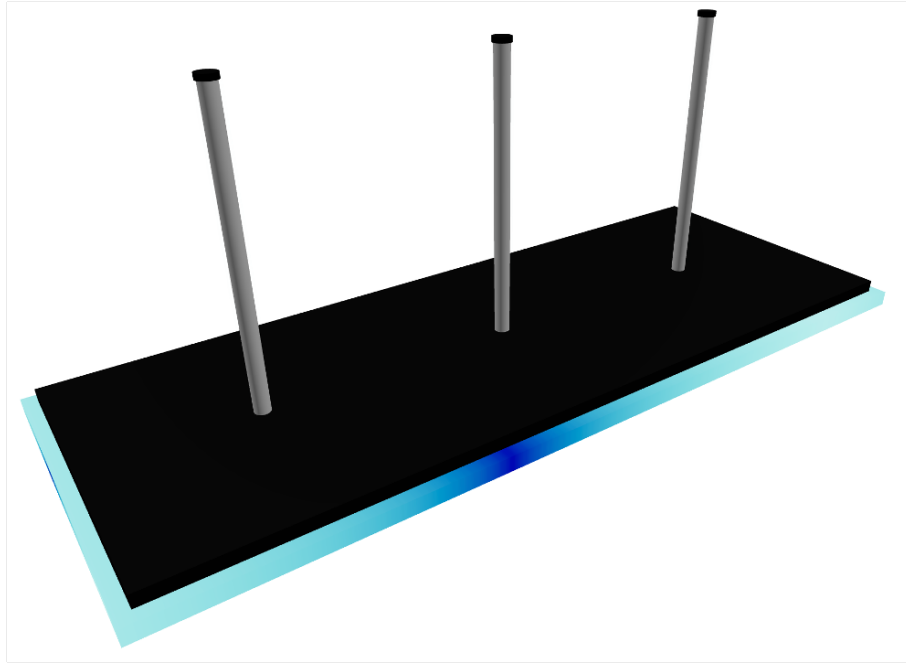


Figure 6: Platform and Pegs

2.2 Choice of Navigation Technique

- **Characterization of the Technique.** For the VR application, the chosen navigation strategy is Physical Locomotion (Room-Scale). This technique maps the user's real-world physical movements (walking, crouching, turning) directly to the virtual camera rig with a 1:1 ratio.
- **Implementation Logic** We rely on the standard WebXRManager provided by Three.js. By enabling XR and adding the VRButton, the system automatically handles the mapping of the headset's physical position to the virtual camera.

```
import { VRButton } from "https://cdn.jsdelivr.net/npm/three@0.161.0/examples/jsm/webxr/VRButton.js";
const renderer = new THREE.WebGLRenderer({ antialias: true, alpha: true });
renderer.setSize(window.innerWidth, window.innerHeight);
renderer.setPixelRatio(window.devicePixelRatio);
renderer.xr.enabled = true;
document.body.appendChild(renderer.domElement);
```

Figure 7: Navigation Technique

2.3 Choice and Design of Selection and Manipulation Techniques

The core gameplay loop requires a robust cycle of Selection, Manipulation, and Placement.

- **Interaction Technique.** We utilize Ray-Casting to interact with objects from a distance. A mathematical ray is projected from the controller's local origin towards its negative Z-axis (forward).

```
function onSelectStart(event) {
  const ctrl = event.target;
  tempMatrix.identity().extractRotation(ctrl.matrixWorld);
  raycaster.ray.origin.setFromMatrixPosition(ctrl.matrixWorld);
  raycaster.ray.direction.set(0, 0, -1).applyMatrix4(tempMatrix);
  const intersects = raycaster.intersectObjects(disks);
```

Figure 8: Interaction Technique

- **Manipulation Technique.** 6-DOF Remote Grabbing We employ Remote Attachment with Offset Preservation. When a disk is grabbed, it is not simply snapped to the controller's center. Instead, we calculate the precise offset between the controller and the object, maintaining that relative distance during movement to mimic a "magnetic beam."

```
const targetWorld = new THREE.Vector3();
controller.localToWorld(targetWorld.copy(localOffset));
tableGroup.worldToLocal(targetWorld);
disk.position.copy(targetWorld);
```

Figure 9: Selection and Manipulation

- **Feedback Mechanisms.** To compensate for the lack of touch, multimodal feedback is triggered immediately upon interaction.

```
disk.material.emissiveIntensity = 2.0;
playGrabSound();
spawnParticles(disk.position, 15, disk.userData.colorHex);
```

Figure 10: Feedback

2.4 Specification of the Virtual Scene

The VR environment is designed to be a "void" to focus attention on the puzzle.

- **Environment.** Dark background (020205), GridHelper, and Reflective Floor.
- **Lighting.** Cinematic setup with Rim Lights (Cyan/Blue).
- **Assets.** High-fidelity Platform, Pegs, and Glass Disks.

- Scenegraph. The scene uses a strict hierarchy. The WorldGroup acts as the master container, which allows us to manipulate the entire game context if necessary (shown in Figure 12).

```
const worldGroup = new THREE.Group();
scene.add(worldGroup);
scene.background = new THREE.Color(0x020205);
scene.fog = new THREE.FogExp2(0x020205, 0.04);

const grid = new THREE.GridHelper(30, 30, 0x0033cc, 0x001133);
worldGroup.add(grid);
const tableGroup = new THREE.Group();
worldGroup.add(tableGroup);
```

Figure 11: Scene hierarchy setup code

2.5 Auditory Feedback Mechanisms

To ensure a responsive and immersive environment, we utilize specific audio cues to communicate game states to the player. These sounds serve as immediate feedback loops, reducing the cognitive load on the player by confirming actions without the need for textual prompts (shown in Figure 13).

The auditory system handles three distinct scenarios:

- Move Confirmation (Tactile Feedback): When a disk is successfully transferred from one tower to another, a concise "click" or "thud" sound is played. This confirms to the user that the input was registered and the move was valid.
- Constraint Violation (Error Alert): If the player attempts an illegal move—specifically, placing a larger disk onto a smaller one—a distinct "buzzer" or warning sound is triggered. This auditory cue reinforces the core rule of the Tower of Hanoi, alerting the player to the error immediately so they can correct their strategy.
- Victory State (Positive Reinforcement): Upon successfully moving the entire stack to the target tower, a celebratory "chime" or fanfare plays. This serves as a reward signal, clearly marking the end of the session and providing a sense of accomplishment.

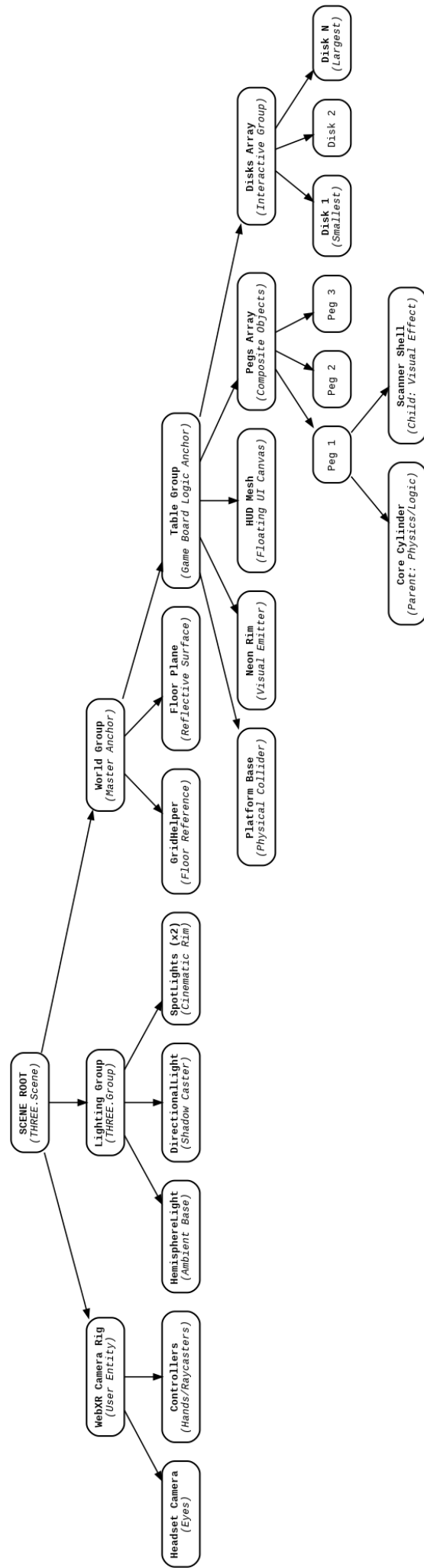


Figure 12: The Scene Graph hierarchy diagram

```

const audioContext = new (window.AudioContext || window.webkitAudioContext)();

function playSound(frequency, duration, type = 'sine') {
  const osc = audioContext.createOscillator();
  const gain = audioContext.createGain();

  osc.frequency.value = frequency;
  osc.type = type; // 'sine' for soft sounds, 'sawtooth' for harsh errors

  osc.connect(gain);
  gain.connect(audioContext.destination);

  osc.start();
  osc.stop(audioContext.currentTime + duration);
}

if (isMoveValid) {
  // SUCCESSFUL TRANSFER
  // Plays a high-pitched 'sine' wave to indicate a smooth drop
  playReleaseSound();

  if (gameWon) {
    // VICTORY STATE
    // Plays a distinct 'triangle' wave chime
    playSound(600, 0.4, 'triangle', 0.5);
  }
} else {
  // INVALID MOVE (Error)
  // Plays a low 'sawtooth' wave (buzzer sound)
  playErrorSound();
}

```

Figure 13: Audio Implementation Logic

2.6 Visual Feedback Mechanisms

In addition to auditory cues, the application employs visual indicators to guide the user through the game logic. These visual states ensure the player is always aware of the current game status and the validity of their interactions (shown in Figure 14).

- **Active Selection State (Highlighting):** When a user clicks or selects a specific disk, it is visually distinguished from the rest of the stack. This is achieved by [insert your specific method here, e.g., adding a glowing border / slightly lifting the disk vertically / changing the disk's brightness]. This visual affordance confirms which element is currently being manipulated.
- **Invalid Move Indication:** To complement the error sound, visual cues prevent the user from completing an illegal move. If a larger disk is dragged over a smaller one, the drop zone may [insert method, e.g., flash red / show a 'prohibited' cursor icon]. If the user releases the mouse, the disk visually "snaps back" to its original tower, clearly rejecting the input.

- Completion Animation: Upon satisfying the winning condition, the interface transitions to a victory state. This is signaled by [insert method, e.g., a victory modal window appearing / the disks flashing in a sequence / a 'Game Over' text overlay], providing a clear visual conclusion to the puzzle.

```
function onSelectStart(event) {
    // ... raycasting logic to find disk ...

    // Visually highlight the selected disk by increasing emission
    // This makes the disk "glow" so the user knows they are holding it
    disk.material.emissiveIntensity = 2.0;
}

function onSelectEnd(event) {
    // Turn off the glow (return to normal state)
    disk.material.emissiveIntensity = 0.4;

    if (isValidMove) {
        // ... move disk to new peg ...

        // CHECK VICTORY CONDITION
        if (pegs[2].disks.length === currentDifficulty) {
            // Trigger Particle Explosion (Visual Reward)
            spawnParticles(new THREE.Vector3(0, 1.5, 0), 100, 0xffff00);
            // Update HUD to show Victory Screen
            drawHUD(finalTime, moveCount, true);
        }
    } else {
        // INVALID MOVE: SNAP BACK
        // Visually reject input by forcing disk back to original coordinates
        disk.position.set(originPeg.x, returnY, originPeg.z);

        // Show floating text alert
        showNotification("Invalid Move!");
    }
}
```

Figure 14: Visual Implementation Logic

3 Implementation of the Virtual Reality and Application

The development of the application followed an incremental approach, starting with a basic geometric prototype and progressively layering interaction logic, VR compatibility, and finally, visual polish.

1. Initial Scene Graph Setup. The first phase focused on establishing the spatial layout and hierarchy without complex visuals.

- Geometry: We utilized standard geometric primitives to represent the puzzle elements. `THREE.BoxGeometry` was used for the table and floor, while `THREE.CylinderGeometry` was used for the pegs and disks.
- Hierarchy: As defined in the design phase, objects were instantiated and immediately added to a parent `tableGroup`. This ensured that the relative transform between the pegs and the platform was locked early in the development process.

```
const tableGroup = new THREE.Group();
worldGroup.add(tableGroup);
const pegGeo = new THREE.CylinderGeometry(0.015, 0.015, 0.6, 32);
const pegMat = new THREE.MeshStandardMaterial({ color: 0xffffff });

for (let i = 0; i < 3; i++) {
  const peg = new THREE.Mesh(new THREE.CylinderGeometry(0.015, 0.015, 0.6, 32), matPegCore);
  peg.position.set(pegPositions[i], 0.3, 0);
  peg.castShadow = !isGhost;
  targetGroup.add(peg);
}
```

Figure 15: Code for generating basic peg geometry

2. Mouse Interaction (Desktop Debugging). Before deploying to the VR headset, the logic for selection and manipulation was tested on a standard desktop screen using mouse interaction. This allowed for rapid debugging of the "Tower of Hanoi" game rules (stacking logic) without the friction of putting on and taking off a headset.

- Navigation. `OrbitControls` was implemented to allow the developer to rotate around the tower and inspect the geometry from all angles.
- Selection. A standard `THREE.Raycaster` was mapped to the mouse coordinates. When the user clicked, a ray was cast from the 2D screen space into the 3D scene to detect intersections with the disk meshes.

```
const raycaster = new THREE.Raycaster();
const mouse = new THREE.Vector2();

window.addEventListener('click', (event) => {
  // Normalize mouse coordinates (-1 to +1)
  mouse.x = (event.clientX / window.innerWidth) * 2 - 1;
  mouse.y = -(event.clientY / window.innerHeight) * 2 + 1;

  raycaster.setFromCamera(mouse, camera);
  const intersects = raycaster.intersectObjects(disks);

  if (intersects.length > 0) {
    // Trigger selection logic
    selectDisk(intersects[0].object);
  }
});
```

Figure 16: Mouse raycasting logic for desktop testing

3. WebXR Setup and VR Interaction Once the game logic was verified on the desktop, the application was migrated to the full WebXR environment.

- Enabling WebXR The renderer was configured to handle the stereoscopic rendering and headset tracking automatically.

```
import { VRButton } from 'three/addons/webxr/VRButton.js';
renderer.xr.enabled = true;
document.body.appendChild(VRButton.createButton(renderer));
```

Figure 17: Enabling WebXR and adding the VR entry button

- Implementing Virtual Raycasting We replaced the Mouse Raycaster with Controller Raycasting.

Controller Setup: We initialized two controllers using `renderer.xr.getController(0)` and `(1)`.

Visual Ray: To help the user aim, a visible line (laser) was attached to the controller, extending along the negative Z-axis.

Event Listeners: The `selectstart` (trigger press) and `selectend` (trigger release) events were bound to the manipulation functions.

```
const controller1 = renderer.xr.getController(0);
controller1.addEventListener('selectstart', onSelectStart);
controller1.addEventListener('selectend', onSelectEnd);
scene.add(controller1);

// The Raycasting logic inside the render loop
function onSelectStart(event) {
    const controller = event.target;
    // Set ray origin/direction to match the physical controller
    tempMatrix.identity().extractRotation(controller.matrixWorld);
    raycaster.ray.origin.setFromMatrixPosition(controller.matrixWorld);
    raycaster.ray.direction.set(0, 0, -1).applyMatrix4(tempMatrix);

    // Check for intersection
    const intersects = raycaster.intersectObjects(disks);
    if (intersects.length > 0) {
        grabObject(intersects[0].object, controller);
    }
}
```

Figure 18: VR controller event listeners and raycasting implementation.

4 Extending to Mixed Reality

The final phase of development involved extending the application to support Mixed Reality (MR). By leveraging the Passthrough capabilities of the headset, the "Tower of Hanoi" puzzle can be brought into the user's real-world environment.

4.1 Roomscale Calibration

Before launching the MR session, the physical play space must be defined.

- Procedure. We utilized the headset's native "Guardian" or "Room Setup" system to calibrate the floor level and define a safe boundary.
- Importance. This step guarantees that the WebXR coordinate system is aligned with the real world. Specifically, it ensures that $y = 0$ in the Three.js scene corresponds exactly to the physical floor, allowing virtual objects to rest naturally on the ground.

4.2 AR/VR Mode Selection

To support both immersion styles, the application was adjusted to handle different XRSession modes: immersive-vr (Opaque) and immersive-ar (Passthrough).

- Technical Implementation: The critical difference lies in the Background Handling.
- In VR Mode: The scene background is set to a dark color (0x020205) to hide the real world.
- In AR Mode: The scene background must be set to null (transparent). This signals the WebXR compositor to render the camera feed behind the virtual objects.

```

import { ARButton } from 'three/addons/webxr/ARButton.js';
import { VRButton } from 'three/addons/webxr/VRButton.js';

// Logic to switch modes
const useAR = true; // Toggle this or use UI to switch

if (useAR) {
  // 1. Create AR Button
  document.body.appendChild(ARButton.createButton(renderer));

  // 2. Set Background to Transparent for Passthrough
  scene.background = null;
} else {
  // 1. Create VR Button
  document.body.appendChild(VRButton.createButton(renderer));

  // 2. Set Background to Void Color
  scene.background = new THREE.Color(0x020205);
}

```

Figure 19: Logic for switching between AR (Passthrough) and VR (Immersive) modes.

4.3 The Placement Phase

In Mixed Reality, we cannot assume the game board should be at (0,0,0) (which might be inside a physical sofa or wall). We introduced a Two-Step Workflow:

1. 1: The Placement State When the application starts, the game is not immediately playable. Instead, the user enters a "Placement Mode."

The Ghost Mesh: A semi-transparent "Ghost" version of the table is instantiated.

Interaction: This ghost mesh follows the user's controller ray (or hovers in front of them). This allows the user to visualize exactly where the tower will spawn in their real room.

Validation: The user pulls the trigger to "Lock" the position.

2. Step 2: The Game State Once the trigger is pulled:

The Ghost Mesh is removed. Then the Real Table (and all disks) is spawned at the exact coordinates of the Ghost. And the physics and game logic are activated.

```

let isPlaced = false;
const ghostTable = createGhostTable(); // Semi-transparent mesh
scene.add(ghostTable);

function onSelectStart(event) {
  if (!isPlaced) {

    // 1. Lock the position
    isPlaced = true;

    // 2. Move the REAL game group to the ghost's position
    worldGroup.position.copy(ghostTable.position);
    worldGroup.rotation.copy(ghostTable.rotation);

    // 3. Hide Ghost, Show Game
    scene.remove(ghostTable);
    worldGroup.visible = true; // Reveal the puzzle

  } else {
    // Normal game logic (Raycasting, Grabbing Disks)
    handleGameInteraction(event);
  }
}
if (!isPlaced) {
  // Make the ghost follow the controller/ray before placement
  updateGhostPosition(controller1);
}

```

Figure 20: Implementation of the "Ghost Mesh" placement phase for Mixed Reality

5 Application Walkthrough

This section describes the standard user journey from launching the application to completing the puzzle, highlighting the interaction metaphors and system states discussed in previous sections.

5.1 The Virtual Reality (VR) Workflow

The VR mode is designed for full immersion, isolating the user from the physical world.

1. Mode Selection: Upon launching the web application, the user is presented with a 2D landing page (shown in Figure 21). The user selects "ENTER VR WORLD" to initiate the immersive session. The background immediately turns dark to establish the virtual environment.



Figure 21: Selection between VR and MR Mode

2. Difficulty Configuration: Before the game begins, a UI menu appears floating in front of the user. The user selects the desired difficulty level by clicking on the number of disks as shown in Figure 22 and 23.

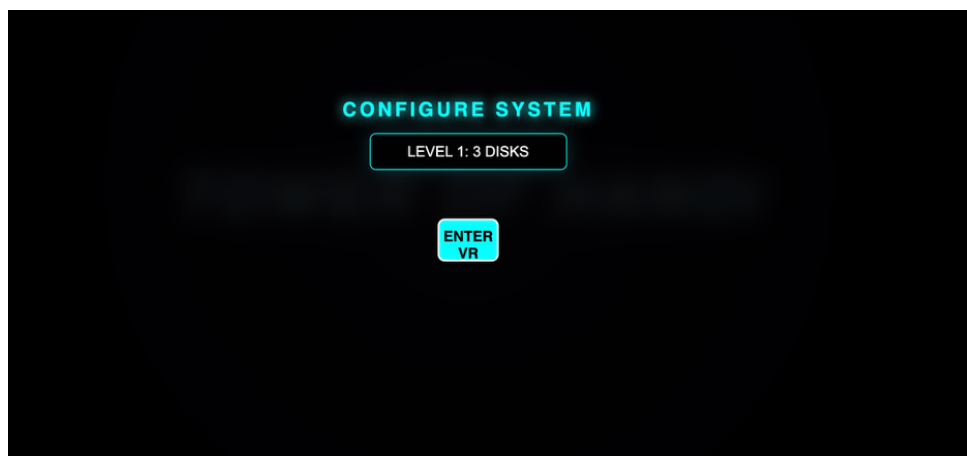


Figure 22: Selection of Difficulty for the number of disks

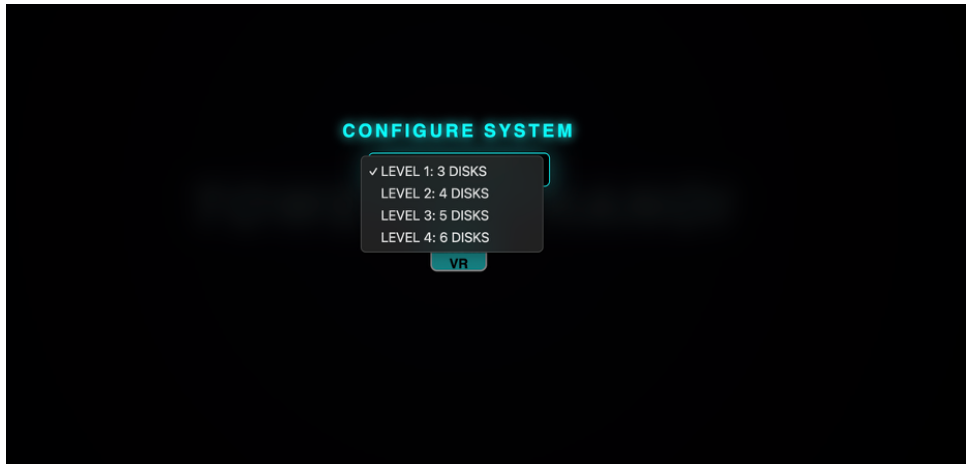


Figure 23: Drop down for selection of Disks

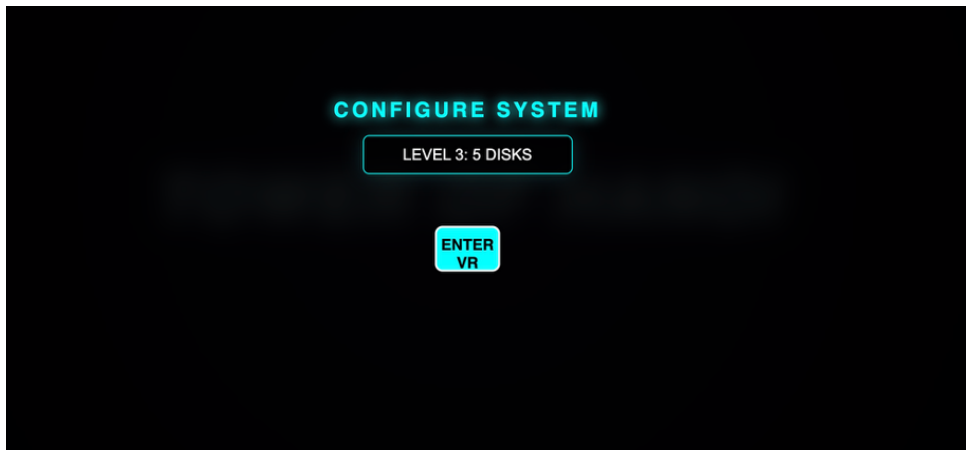


Figure 24: Selected number of disks to be played

3. Gameplay Initialization: Once the difficulty is confirmed, the game board spawns, and the system timer begins counting up. For this example, a 5-disk configuration is generated on the left peg (shown in Figure 25).

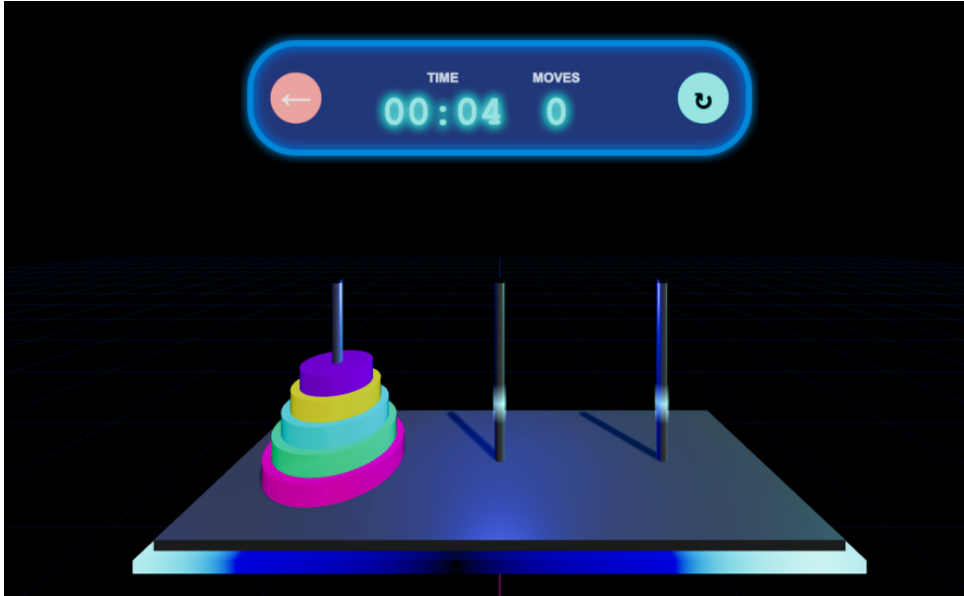


Figure 25: Start of the Game

4. Interaction Audio-Visual Feedback: The user plays by selecting and moving disks using the ray-casting technique. The system provides continuous feedback for every action:

Grab: When the user holds the trigger, a "rising" tone plays to confirm the disk is held.

Placement (Valid Move): Every time the user successfully releases a disk onto a new peg, a distinct "clack" or placement sound plays to confirm the move was registered.

Invalid Move: If the user attempts to place a larger disk on a smaller one, a "buzzer" error sound plays, the text "Invalid Move" flashes on the HUD, and the disk snaps back to its original position (shown in Figure 26).

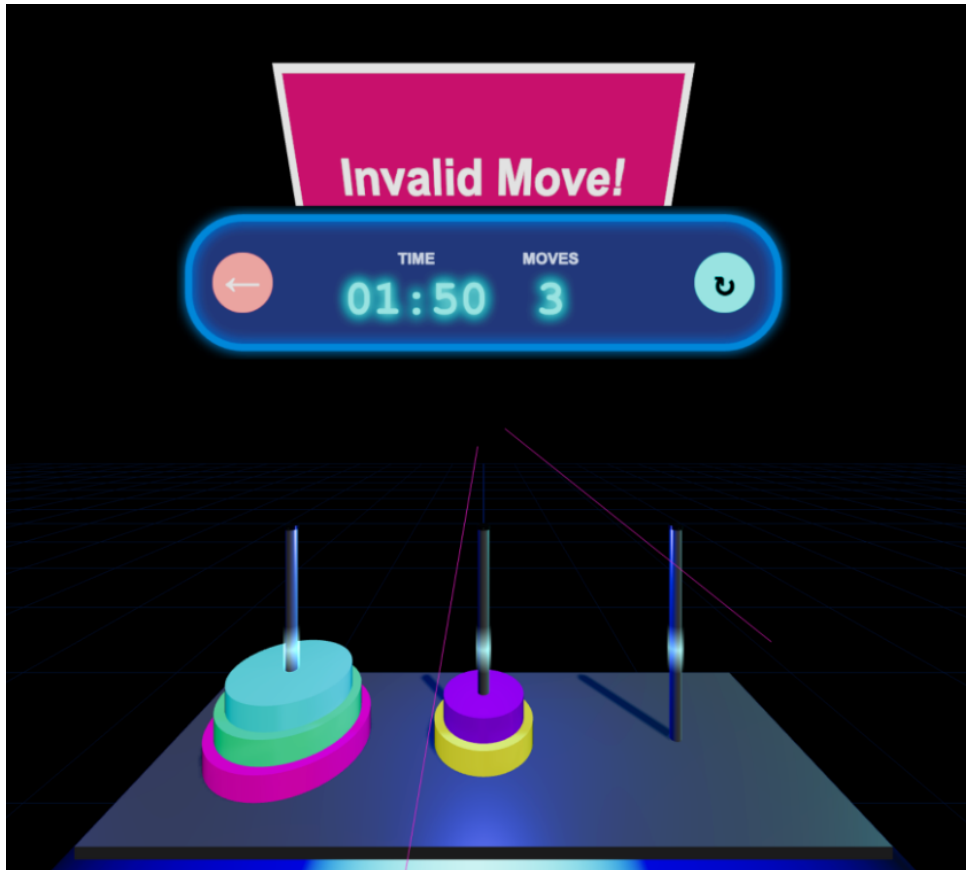


Figure 26: Invalid Move Message for Error

5. Completion State: Upon successfully moving the entire stack to the target peg, the system detects the win condition. The timer stops, a "Victory!" message appears, and visual effects (particles) are triggered to reward the user (shown in Figure 27).



Figure 27: Victory Message upon completion of the game

5.2 The Mixed Reality (MR) Workflow

The MR mode integrates the game into the user's real-world environment using Passthrough technology.

1. Mode Selection: From the landing page, the user selects "BLEND WITH REALITY (MR)". The headset's cameras activate, replacing the black background with a live feed of the physical room.



Figure 28: Selection between VR and MR Mode

2. Difficulty Configuration: Similar to the VR mode, the user selects the disk count (difficulty level) via the floating UI (shown in Figure 29).

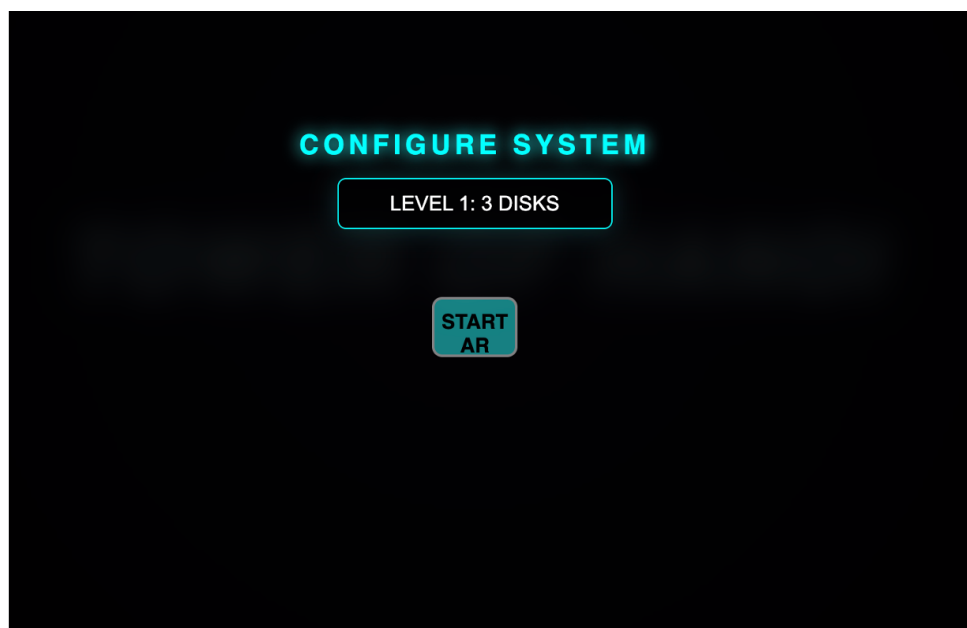


Figure 29: Selection for the number of Disks

3. Spatial Anchoring (Placement Phase): This is the key step unique to Mixed Reality.

Ghosting: A semi-transparent "Ghost Table" appears, following the user's controller or gaze.

Placement: The user physically identifies a suitable surface (e.g., a real desk or an empty floor spot) and aligns the ghost mesh with it.

Locking: Pressing the trigger locks the digital table to that physical coordinate, ensuring the game stays fixed in the real world.

4. Contextual Gameplay: The solid game board spawns at the locked location, and the timer starts as shown in Figure 30. The user plays the game as normal, with the same rules and feedback loops as the VR mode:

Auditory Confirmation: Just as in VR, every valid placement generates a sound, giving the user confidence that the digital object has "landed" on the virtual peg despite the lack of physical touch.



Figure 30: Starting of the Game using Mixed Reality

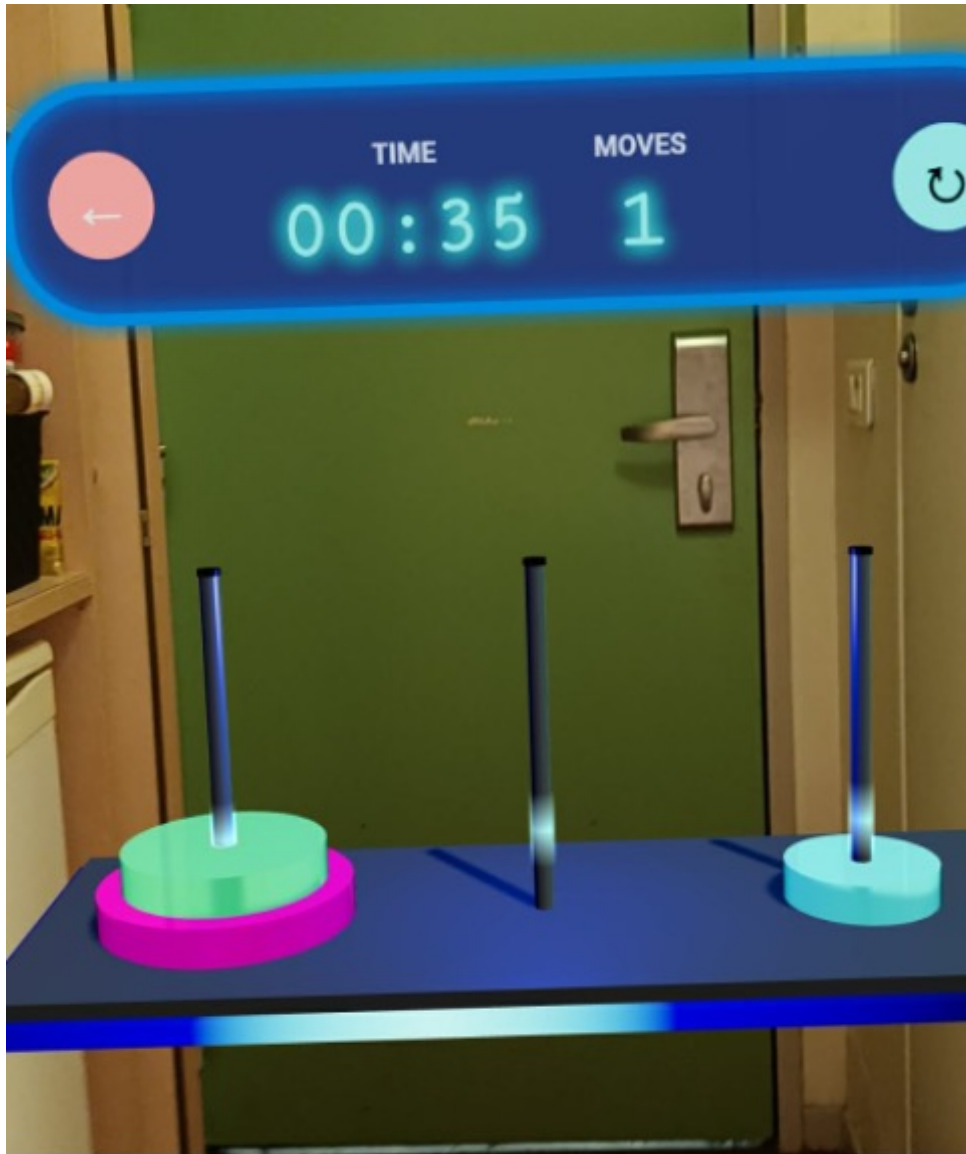


Figure 31: Tower of Hanoi Game in MR

5. Completion State: When the puzzle is solved, the Victory message and effects appear, overlaid onto the real-world view.



Figure 32: Invalid Message for Error Move



Figure 33: Completion of the game

6 Conclusion

The development of the Virtual Reality and Mixed Reality Tower of Hanoi application successfully demonstrates the capability of WebXR to deliver immersive, interactive experiences directly through a browser. By following an incremental development strategy, we achieved a robust system that effectively bridges the gap between digital puzzles and physical space.

The project successfully established a modular scene graph architecture that separates visual fidelity from physical logic, enabling complex effects like PBR glass and neon emissions without compromising performance. We implemented an intuitive Ray-Casting interaction model utilizing "Remote Grabbing" with offset preservation. This system, reinforced by immediate auditory and visual feedback for every move, effectively compensated for the lack of tactile response and ensured a satisfying user experience.

Additionally, the integration of Mixed Reality capabilities expanded the application beyond a static virtual environment. By leveraging Passthrough technology and a custom "Ghost Table" placement workflow, users can spatially anchor the game board to their real-world surroundings. Combined with natural Room-Scale locomotion, this approach

eliminates motion sickness and demonstrates the flexibility of WebXR in creating both fully immersive and augmented experiences.

While the current prototype is fully functional, future iterations could enhance the experience further. Implementing hand-tracking would allow users to grab disks directly with their fingers, creating a "virtual hand" metaphor that offers even greater naturalism. Additionally, multi-user synchronization could transform this solitary puzzle into a collaborative or competitive social experience.